02 / ALGEBRA / EX25

Pseudocode guide

From a mathematical problem to an algorithm and then a program.

DOCUMENT 04 / PSEUDOCODE

What you will learn

- Turn a mathematical problem into clear sequential steps.
- Identify inputs, calculations, outputs, and checks.
- Write pseudocode before implementing NumPy operations.

Prerequisites: EX25 theory, reference, and exercise guides.

Reading order: 1. Theory → 2. Reference → 3. Exercise → 4. Pseudocode → 5. Programming.

1. What is pseudocode?

Pseudocode is a **structured, readable description of an algorithm**. It is not executable Python and need not follow Python syntax. It separates the logic of the problem from the details of implementation.

> **Rule:** first decide what to calculate, then establish the sequence, and only then choose Python operations. Do not search for code before understanding the mathematics.

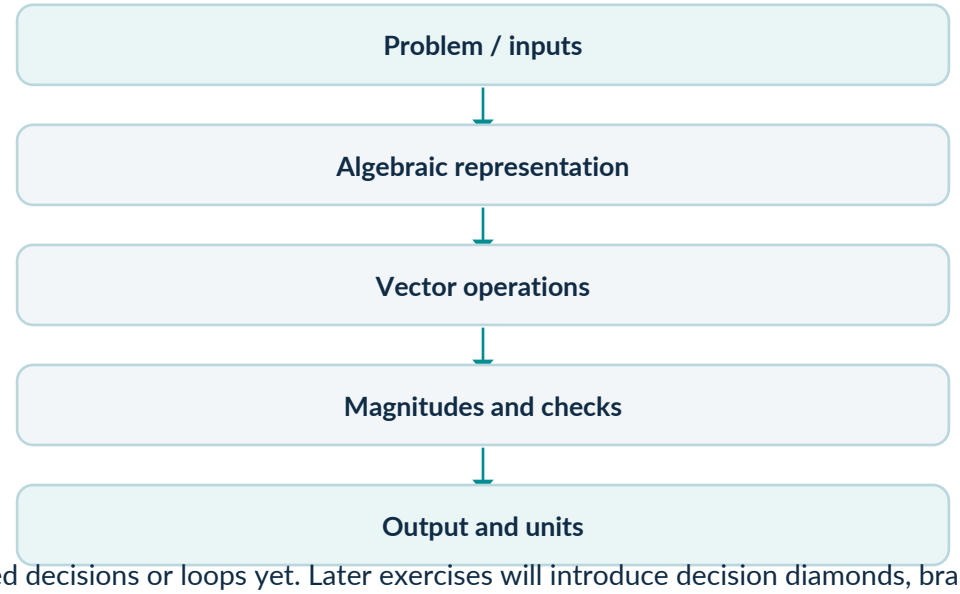
2. Five questions before writing any algorithm

- **What is known?** Inputs, units, dimensions, and reference frame.
- **What is required?** Specific vector and scalar outputs.
- **Which mathematics connects them?** Equations, properties, and constraints.
- **What steps are needed?** Ordered operations.
- **How will I check?** Manual example, units, signs, and tolerances.

3. Mathematics → algorithm → Python

Mathematics	Pseudocode	Python tool
Vector (a,b)	CREATE two-component vector	np.array(...)
Sum u+v	ADD components	u + v
2-norm	COMPUTE length	np.linalg.norm(...)
Magnitude + unit	DISPLAY result	print(...)

4. Conceptual flow diagram



We do not need decisions or loops yet. Later exercises will introduce decision diamonds, branches, iteration arrows, and convergence checks.

5. Solved pseudocode: Part A

This algorithm follows the fully worked algebra example from the exercise.

```
START
  DEFINE u = (3, -1) and v = (-2, 4)
  CREATE two numeric vectors
  COMPUTE sum = u + v
  COMPUTE combination = 2 * u - v
  COMPUTE norm_of_sum
  COMPUTE norm_of_combination
  DISPLAY the results
  COMPARE with manual calculations
END
```

6. Your pseudocode: Part B, forces

Complete the following outline independently. You must decide which operation belongs at each stage.

```
START
  DEFINE the given forces F1 and F2
  CREATE their component vectors
  COMPUTE ... (first force magnitude)
  COMPUTE ... (second force magnitude)
  COMPUTE ... (resultant vector)
  COMPUTE ... (resultant magnitude)
  DISPLAY ... (with units)
  CHECK ... (against hand calculations)
END
```

7. Pseudocode quality checklist

- Every step has one understandable purpose.
- Intermediate values are available before later steps use them.
- Vector quantities and scalar magnitudes are clearly distinguished.
- Input and output units are consistent.
- There is a check independent of the code.

8. Common mistakes

Too vague: “calculate forces”. This does not define the operation.

Too much Python syntax: copying NumPy commands without knowing their meaning.

No validation: printing a number without checking whether it makes sense.

9. Template for future exercises

```
START
  IDENTIFY inputs, units, and dimensions
  DEFINE variables and representations
  APPLY mathematical relationships
  COMPUTE intermediate values
  VALIDATE units, signs, and outputs
  DISPLAY / SAVE the results
END
```

Later, this template will grow to include **IF / THEN**, **FOR**, **WHILE**, input validation, and reusable modules, always with a clear explanation of why each construct is necessary.